

Cahier des charges — E-enseignement

Plateforme pédagogique et vitrine du groupe : cours, exercices corrigés et documentation

Équipe	Moncef MALLEM · Christophe ALEIXO · Alexis DE JESUS · Thomas MONTOUT
Épreuve	E5 – BTS SIO option SLAM, 2 ^e année
Projet	E-enseignement
Établissement	Paris Ynov Campus – Nanterre
Enseignant encadrant	M. Okacha BEN AHMED DAHO
Stack technique	HTML5 · CSS3 · JavaScript · highlight.js · Docker (nginx)
Dépôt	github.com/FlytziTv/e5-website
Année	2026 – 2027

Sommaire

- I. Présentation du Projet
- II. Analyse des Besoins et Périmètre
- III. Description Graphique et Ergonomique
- IV. Architecture Technique
- V. Structure et Dictionnaire de Données
- VI. Description Fonctionnelle
- VII. Qualité, Accessibilité et Sécurité
- VIII. Tests et Validation
- IX. Organisation et Répartition du Travail
- X. Bilan et Perspectives
- XI. Annexes

I. Présentation du Projet

1. Contexte

Dans le cadre de l'épreuve E5 du BTS SIO option SLAM, nous avons développé en équipe de quatre le site E-enseignement, du 11 au 25 septembre 2026. Ce projet vise à valider notre maîtrise de la conception d'un site web statique, de l'intégration HTML / CSS et de la programmation JavaScript côté client, en travaillant de manière collaborative avec Git et GitHub.

2. La Vision et les Objectifs

E-enseignement est un site à double usage : une plateforme pédagogique pour apprendre les bases de la programmation, et une vitrine qui présente les membres du groupe et leurs entreprises d'accueil. Son parti pris : partir de l'algorithme, puis montrer sa traduction dans plusieurs langages, pour comprendre le raisonnement une fois et l'écrire ensuite dans n'importe quel langage.

Les objectifs principaux du projet sont :

- Proposer des cours progressifs, de la structure d'un programme jusqu'aux fichiers
- Faire pratiquer avec des exercices corrigés, chacun écrit en algorithme, C, Java, JavaScript et PHP
- Présenter le groupe aux recruteurs : postes, contrats, CV et profils LinkedIn
- Rendre la documentation du projet consultable directement depuis le site
- Garantir un site léger, lisible sur tout support et utilisable sans installation

3. Périmètre livré

Le site livré compte quatre pages : Accueil, Cours, Exercices et Documentation, reliées par un menu commun. Il est entièrement statique : aucun serveur applicatif ni base de données, ce qui permet de l'ouvrir directement dans un navigateur, depuis XAMPP ou dans un conteneur Docker. Les cours fonctionnent en HTML et CSS seuls ; JavaScript est réservé à la page Exercices et à la coloration du code. L'espace membre, l'exécution réelle du code et le formulaire de contact ne font pas partie du périmètre.

II. Analyse des Besoins et Périmètre

1. Acteurs du système

Acteur	Description
Étudiant	Cible principale. Consulte les cours, choisit un exercice et un langage, lit le corrigé et affiche le résultat attendu.
Recruteur / entreprise	Consulte les fiches des membres, ouvre leurs CV et leurs profils LinkedIn, repère les entreprises d'accueil.
Enseignant / jury	Évalue le site et consulte les livrables du projet depuis la page Documentation.
Services externes	Fournisseurs sollicités au chargement des pages : highlight.js (cdnjs) pour la coloration du code et Google Fonts pour les polices.

Tous les visiteurs ont les mêmes droits : le site ne comporte ni compte ni authentification.

2. Besoins fonctionnels

- **Navigation commune** : menu latéral identique sur les quatre pages, avec la page active mise en évidence.
- **Présentation du groupe** : fiches des quatre membres (poste, alternance ou stage, CV, LinkedIn), langages pratiqués et entreprises d'accueil.
- **Cours** : dix notions consultables une par une, chacune décrite en algorithme puis traduite en C, JavaScript et PHP.
- **Exercices** : six exercices corrigés, consultables dans cinq écritures, avec énoncé, résultat attendu, code coloré et simulation d'exécution.
- **Documentation** : accès aux livrables du projet (cahier des charges, charte graphique, Gantt, cas d'utilisation, maquette) et aux outils utilisés.

3. Besoins non fonctionnels

- **Ergonomie** : interface claire, responsive de 360 à 1920 px, navigation en deux clics maximum vers n'importe quel contenu.
- **Performance** : site statique sans étape de compilation, scripts chargés en bas de page, chargement en moins de 3 secondes.

- **Robustesse** : cours utilisables sans JavaScript ; sans connexion, polices de remplacement et code non coloré mais lisible.
- **Accessibilité** : HTML sémantique, attributs ARIA, contraste WCAG AA, respect du réglage « réduire les animations ».
- **Maintenabilité** : variables CSS centralisées, une feuille de style par page, données des exercices séparées du HTML, code commenté en français.

III. Description Graphique et Ergonomique

1. Thème et Design

L'interface adopte un thème clair, violet et lavande, sans ombre ni dégradé : la hiérarchie vient du contraste et de l'espacement. Le titre du site, « > e-enseignement », est écrit en police monospace avec un curseur clignotant, comme une invite de terminal, pour rappeler l'univers du code. Les blocs de code utilisent un thème sombre (atom-one-dark) qui ressort sur le fond clair.

Couleur	Code	Usage
Violet — primaire	#5B21B6 (survol #4C1D95)	Boutons principaux, liens, page active, curseur du titre
Citron — accent	#D9F24B	Badges et bouton d'action principal, toujours avec un texte foncé
Encre	#14121F	Titres, texte fort, pied de page
Lavande	#F3F0FA	Sections alternées, étiquettes, zones de fond
Blanc et bordure	#FFFFFF · #E7E1F5	Fond par défaut, cartes, séparateurs
Texte secondaire	#4B5563	Descriptions, rôles, légendes

- **Typographies** : Space Grotesk pour les titres et le texte courant ; IBM Plex Mono pour le code, les étiquettes et le titre du site. H1 60 px, H2 34 px, H3 19 px, texte courant 16 px.
- **Espacements et formes** : échelle 4 · 8 · 12 · 20 · 32 · 48 · 72 px, boutons en pilule, cartes arrondies à 14 px.

Toutes ces valeurs sont déclarées une seule fois sous forme de variables CSS dans `css/global.css`.

2. Composants UI et Accessibilité

- **Icônes SVG** : icônes au trait dessinées directement dans le HTML (menu, avatars, entreprises, Instagram), visibles sans JavaScript et sans fichier externe.
- **Cartes** : composant commun aux fiches membres, entreprises et documents, avec effet de survol.
- **Boutons** : variantes principale, accent et secondaire, hauteur minimale de 44 px.
- **Animations CSS** : transitions au survol, apparition des aperçus de cours en 0,3 s, curseur clignotant ; ces deux dernières sont coupées lorsque le réglage `prefers-reduced-motion` est actif.

3. Principales interfaces développées

- **Gabarit commun** : barre du haut (logo Ynov et titre), menu latéral « Navigation » avec icônes, pied de page (logo, adresse, Instagram).
- **Accueil** : texte de présentation, rubriques « Le groupe », « Les langages » (SQL, PHP, Java, C, JavaScript, HTML / CSS) et « Les entreprises » (Snasarl, Helpline, óteam, Coface).
- **Cours** : liste numérotée des dix notions et panneau d'aperçu qui s'ouvre au clic.
- **Exercices** : boutons de langage, énoncé et résultat attendu à gauche, éditeur de code à droite (nom du fichier, bouton Executer, zone Sortie), liste des exercices en dessous.
- **Documentation** : grille de cartes des livrables (type PDF ou PNG, titre, description, lien « Ouvrir ») et bloc « Outils du projet ».

IV. Architecture Technique

Le site est entièrement statique : chaque page est un fichier HTML servi tel quel, sans serveur applicatif, sans base de données et sans étape de compilation. Toute la logique s'exécute dans le navigateur, ce qui permet d'héberger le site sur n'importe quel serveur web.

1. Stack Technologique

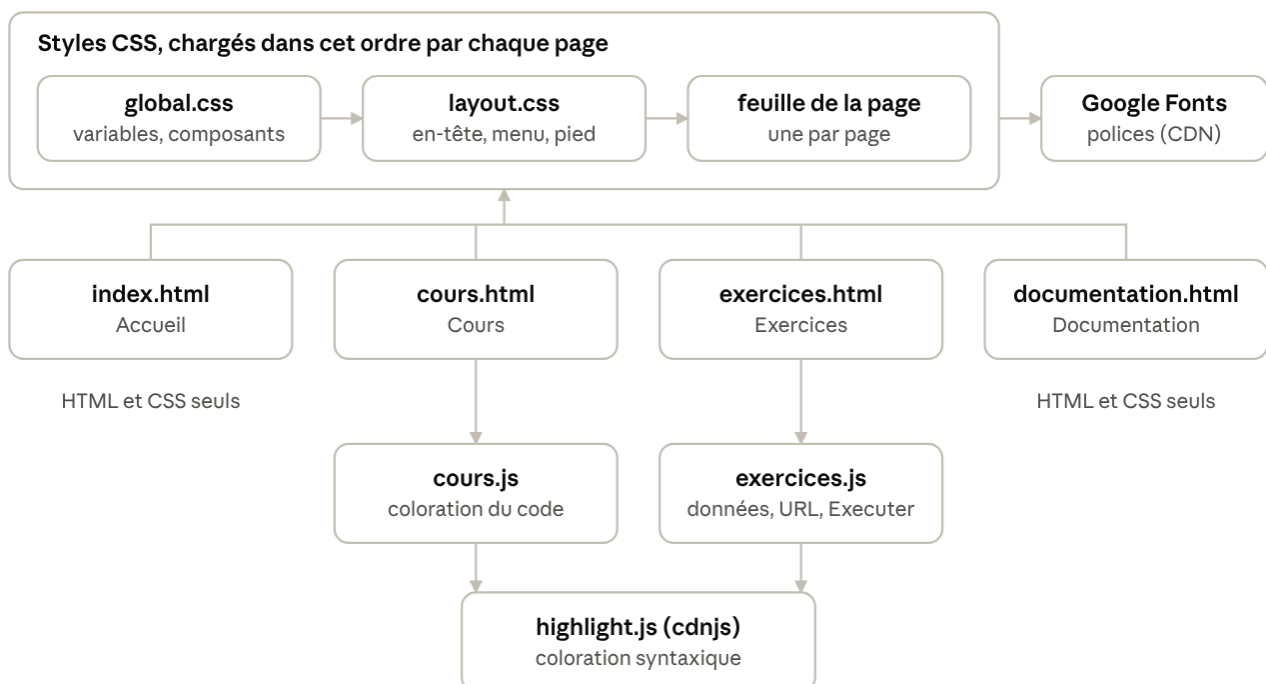
Technologie	Type	Rôle dans le projet
HTML5	Structure	Pages sémantiques (<code>header</code> , <code>nav</code> , <code>aside</code> , <code>main</code> , <code>section</code> , <code>article</code> , <code>footer</code>), icônes SVG intégrées

Technologie	Type	Rôle dans le projet
CSS3	Mise en forme	Variables, Flexbox, Grid, <code>@keyframes</code> , transitions, sélecteur <code>:checked</code> pour le dépliage des cours, media queries
JavaScript (ES6)	Interactivité	Page Exercices : lecture de l'URL, affichage des données, bouton Executer, génération de la liste
highlight.js 11.10	Bibliothèque	Coloration syntaxique du C, Java, JavaScript, PHP et d'un langage « algo » défini par l'équipe
Google Fonts	Polices	Space Grotesk et IBM Plex Mono
Docker / nginx:alpine	Serveur local	Conteneur e-enseignement qui sert le site sur le port 6969, fichiers montés en lecture seule
Git / GitHub	Versionnement	Branches par fonctionnalité, pull requests, historique partagé

2. Schéma de l'Architecture

Un site statique : HTML, trois niveaux de CSS et JavaScript ciblé

Aucun serveur applicatif ni base de données : nginx, XAMPP ou le navigateur servent les fichiers tels quels



architecture du site · 4 pages, 3 niveaux de CSS

Chaque page charge les trois niveaux de styles ; seules les pages Cours et Exercices chargent du JavaScript, qui s'appuie sur highlight.js pour colorer le code.

3. Organisation des fichiers

Dossier / fichier	Contenu
<code>index.html</code>	Page d'accueil
<code>pages/</code>	<code>cours.html</code> · <code>exercices.html</code> · <code>documentation.html</code>
<code>css/</code>	<code>global.css</code> (variables, composants) · <code>layout.css</code> (barre du haut, menu, pied de page) · <code>accueil.css</code> , <code>cours.css</code> , <code>exercices.css</code> , <code>documentation.css</code>
<code>js/</code>	<code>script.js</code> (commun) · <code>exercices.js</code> (données et affichage des exercices) · <code>cours.js</code> (coloration du code)
<code>images/icons/</code>	Logo Ynov en SVG
<code>documents/</code>	CV des membres (<code>cv-prenom-nom.pdf</code>) et dossier <code>Documentation/</code> des livrables
<code>README.md</code>	Présentation, lancement, structure et conventions
<code>docker-compose.yml</code>	Lancement du site avec Docker

Fichiers nommés en minuscules, sans accent ni espace, mots séparés par un tiret. Code, classes CSS et commentaires en français.

V. Structure et Dictionnaire de Données

Le site ne possède pas de base de données. Le contenu de la page Exercices est stocké dans deux objets JavaScript du fichier `js/exercices.js` ; la page HTML ne contient qu'un gabarit vide que le script remplit au chargement. Ajouter un exercice revient à ajouter une entrée dans l'objet `exercices` , sans toucher au HTML.

1. Objet `langages`

Clé	Champ	Type	Description
<code>Algo</code> , <code>C</code> , <code>Java</code> , <code>JavaScript</code> , <code>PHP</code>	<code>extension</code>	Chaîne	Extension affichée dans le nom du fichier (<code>Ex01.c</code> , <code>Ex01.php</code> ...)

Clé	Champ	Type	Description
	coloration	Chaîne	Nom du langage transmis à highlight.js (php-template pour le PHP mêlé à du HTML)

Les clés doivent être identiques aux attributs data-language des boutons du HTML, majuscules comprises.

2. Objet exercices

Champ	Type	Description
Clé (1 , 2 ...)	Nombre	Numéro de l'exercice, utilisé dans l'URL (exo=2)
numero	Chaîne	Numéro affiché sur deux chiffres ("01")
nom	Chaîne	Titre de l'exercice
enonce	Chaîne (HTML)	Énoncé, pouvant contenir des balises <code>
attendu	Chaîne	Résultat que le programme doit afficher, repris dans la zone Sortie
codes	Objet	Un corrigé par langage : Algo , C , Java , JavaScript , PHP

3. Contenu actuel

N°	Exercice	Notions travaillées
01	Surface et périmètre d'un rectangle	Saisie, conversion en nombre, calcul
02	Équation du premier degré	Conditions imbriquées, cas particuliers
03	Diviseurs d'un nombre	Boucles, modulo
04	Procédure mesDiviseurs	Procédures et paramètres
05	Prix minimum, maximum et moyen	Tableaux, parcours, moyenne
06	Copier-coller un fichier	Lecture et écriture de fichiers

VI. Description Fonctionnelle

1. Pages et adresses

Page	Adresse	Contenu
Accueil	<code>index.html</code>	Présentation, le groupe, les langages, les entreprises
Cours	<code>pages/cours.html</code>	Dix notions de programmation, de l'algorithme à sa traduction
Exercices	<code>pages/exercices.html?language=C&exo=2</code>	Exercice et langage choisis ; sans paramètre, exercice 1 en algorithme
Documentation	<code>pages/documentation.html</code>	Livrables du projet et outils utilisés

Chaque page charge `global.css` , puis `layout.css` , puis sa propre feuille de style. Cet ordre permet à une page d'ajuster un style commun sans recourir à `!important` .

2. Cours : dépliage sans JavaScript

La page Cours affiche un seul aperçu à la fois, sans aucune ligne de JavaScript :

1. Dix boutons radio invisibles partagent le même attribut `name` : un seul peut être coché à la fois.
2. Chaque ligne de la liste est un `<label>` relié à son bouton radio : cliquer dessus le coche.
3. Le sélecteur CSS `#cours-N:checked ~ .cours-panneaux #panneau-N` met la ligne en évidence et affiche le panneau correspondant, avec une animation d'apparition de 0,3 s.

Les caractères `<` , `>` et `&` des exemples de code sont échappés (`<` , `>` , `&`) pour être affichés et non interprétés par le navigateur. `cours.js` se limite à colorer les blocs de code.

3. Exercices : affichage piloté par les données

Au chargement de la page, `exercices.js` :

1. Lit le langage et le numéro d'exercice dans l'URL (`URLSearchParams`), avec des valeurs par défaut.
2. Remplit le numéro, le titre, l'énoncé, le résultat attendu et le nom du fichier (`Ex02.c`).

3. Insère le corrigé dans `<pre><code>` puis le colore avec highlight.js, après avoir déclaré les mots-clés du pseudo-code (`ALGORITHME` , `DÉBUT` , `SI` , `LIRE` ...).
4. Active le bouton du langage courant ; un clic sur un autre langage recharge la page en conservant le même exercice.
5. Génère la liste des exercices, chaque entrée étant un lien vers son adresse.
6. Au clic sur « Executer », affiche le résultat attendu dans la zone Sortie.

L'exécution est simulée : un navigateur ne peut pas exécuter du C, du Java ou du PHP. Le bouton montre ce que le programme doit afficher, afin que l'étudiant compare avec sa propre exécution.

4. Documentation et présentation du groupe

- Les boutons « Voir le CV » ouvrent les PDF du dossier `documents/` ; les liens LinkedIn et Instagram s'ouvrent dans un nouvel onglet.
- Les cinq livrables de la page Documentation s'ouvrent dans un nouvel onglet, d'où ils peuvent être téléchargés.
- Le `.gitignore` exclut les brouillons du dossier `documents/` mais publie les CV et les livrables, nécessaires au fonctionnement de ces liens en ligne.

VII. Qualité, Accessibilité et Sécurité

1. Sécurité du code client

- **Injection de contenu** : les textes sont insérés avec `textContent` , qui affiche les caractères sans les interpréter. `innerHTML` n'est utilisé que pour l'énoncé et la liste des exercices, dont le contenu provient de nos propres données et jamais d'une saisie utilisateur.
- **Paramètres d'URL** : le langage et le numéro lus dans l'adresse sont contrôlés ; une valeur inconnue est remplacée par la valeur par défaut. Les liens sont construits avec `URLSearchParams` , qui encode les caractères spéciaux.
- **Liens externes** : ouverts avec `target="_blank"` et `rel="noopener"` , pour que la page ouverte ne puisse pas agir sur l'onglet du site.
- **Données personnelles** : aucun formulaire, aucun cookie, aucune donnée collectée. Les CV sont publiés avec l'accord de chaque membre.
- **Serveur Docker** : les fichiers du projet sont montés en lecture seule dans le conteneur nginx.

2. Robustesse

- Si highlight.js ne se charge pas (pas de connexion), le test `typeof hljs !== "undefined"` évite toute erreur : le code s'affiche simplement sans couleurs.
- Sans Google Fonts, des polices de remplacement (`sans-serif` , `monospace`) prennent le relais.
- Les cours, la navigation, l'accueil et la documentation restent utilisables avec JavaScript désactivé.

3. Accessibilité

- Structure sémantique et titres hiérarchisés h1 → h3 sur chaque page.
- `alt` sur les images, `aria-label` sur les liens réduits à une icône, `aria-hidden` sur les SVG décoratifs, `aria-current="page"` sur la page active.
- Contraste conforme WCAG AA ; texte foncé obligatoire sur la couleur citron.
- Animations coupées lorsque `prefers-reduced-motion` est actif.
- Zones cliquables d'au moins 44 px et curseur adapté aux éléments cliquables.

4. Qualité et maintenabilité

- Valeurs de couleurs, tailles et espacements centralisées dans les variables de `global.css`.
- Une feuille de style par page, chargée après les styles communs.
- Code commenté en français pour expliquer chaque choix technique.
- Messages de commit au format `type(portée): message` , par exemple `feat(exercices): add exercises 04 to 06`.

VIII. Tests et Validation

1. Méthode de vérification

- **Validation du code** : passage des pages au validateur HTML du W3C.
- **Responsive** : mode appareil des outils de développement du navigateur, en 360, 768 et 1920 px de large.
- **Robustesse** : tests avec JavaScript désactivé et sans connexion internet.
- **Navigateurs** : Chrome, Firefox, Edge et Safari.
- **Environnements** : ouverture directe du fichier, serveur XAMPP et conteneur Docker.

2. Tests Fonctionnels Manuels

Scénario testé	Résultat attendu	Statut
Navigation entre les 4 pages	Menu présent sur chaque page, page active en violet	À tester
Clic sur « Voir le CV » d'un membre	Le PDF du bon membre s'ouvre	À tester
Ouverture des 10 cours avec JavaScript désactivé	Chaque aperçu se déploie, un seul à la fois	À tester
Ouverture de <code>exercices.html</code> sans paramètre	Exercice 1 affiché en algorithmme	À tester
Adresse <code>exercices.html?language=PHP&exo=5</code>	Exercice 5 affiché en PHP, bouton PHP actif	À tester
Adresse avec un langage ou un numéro inconnu	Retour aux valeurs par défaut, sans erreur	À tester
Changement de langage sur un exercice	Même exercice affiché dans le nouveau langage	À tester
Clic sur « Executer »	Résultat attendu affiché dans la zone Sortie	À tester
Chargement sans connexion internet	Code lisible sans couleur, polices de remplacement	À tester
Ouverture des 5 livrables de la Documentation	Chaque fichier s'ouvre dans un nouvel onglet	À tester
Affichage en 360 px de large	Menu au-dessus du contenu, une colonne, aucun défilement horizontal	À tester
<code>docker compose up -d</code>	Site disponible sur <code>http://localhost:6969</code>	À tester

IX. Organisation et Répartition du Travail

1. Méthodologie et outils

- **Trello** : suivi des tâches et répartition entre les membres.
- **Figma** : maquette des pages avant le développement.
- **Mermaid** : diagramme de cas d'utilisation.

- **GanttProject** : planification des lots.
- **VS Code** : éditeur commun, avec l'extension Live Server.
- **Git et GitHub** : une branche par fonctionnalité (`page/accueil` , `page/exercices` , `style/global` , `document` ...), fusion dans `main` par pull request.

2. Planning

Le projet s'est déroulé du 11 au 25 septembre 2026, en six lots (diagramme de Gantt en annexe).

Lot	Dates	Tâches	Livrable
1. Cadrage et gestion	11/09	Cahier des charges, tableau Trello, création du dépôt	Cahier des charges, dépôt GitHub
2. Conception et design	14/09 – 15/09	Maquette, charte graphique, diagramme de cas d'utilisation	Maquette, charte, diagrammes
3. Socle technique	15/09 – 21/09	Styles communs, barre du haut, menu, pied de page, environnement Docker	Gabarit commun aux 4 pages
4. Modules fonctionnels	21/09 – 24/09	Pages Accueil, Cours, Exercices et Documentation	Site fonctionnel
5. Contenus pédagogiques	24/09 – 25/09	Rédaction des 10 cours et des 6 exercices	Cours et exercices en ligne
6. Recette et livraison	25/09	Tests, README, préparation de la soutenance	Site livré, documentation

3. Réalisations de chaque membre

Membre	Réalisations principales
Alexis DE JESUS	Création du dépôt et de l'arborescence de base ; mise en place de <code>global.css</code> et <code>layout.css</code> (structure commune et navigation)
Christophe ALEIXO	Environnement Docker (<code>docker-compose.yml</code> , <code>nginx</code>) et premier README ; page Documentation (HTML, CSS et contenus) ; correction des liens entre pages
Moncef MALLEM	Intégration HTML / CSS des pages Accueil et Cours

Membre	Réalisations principales
Thomas MONTOUT	Page Exercices et logique <code>exercices.js</code> ; rédaction des cours et des exercices ; uniformisation du style des pages (typographie, cartes, animations, titre façon terminal) ; README final et <code>.gitignore</code>

X. Bilan et Perspectives

1. Compétences validées

Le développement d'E-enseignement nous a permis de valider des compétences concrètes de développement web :

- Intégration HTML5 sémantique et CSS3 à partir d'une maquette et d'une charte graphique.
- Organisation d'une feuille de style maintenable : variables, styles communs, styles par page.
- Interactions sans JavaScript grâce aux sélecteurs CSS (`:checked` , `:hover`) et aux animations.
- Manipulation du DOM en JavaScript : lecture de l'URL, insertion de contenu, gestion des événements.
- Séparation des données et de l'affichage pour faire évoluer le contenu sans toucher au HTML.
- Intégration d'une bibliothèque tierce (`highlight.js`) et extension avec un langage personnalisé.
- Conteneurisation d'un site avec Docker Compose et nginx.
- Travail collaboratif avec Git : branches, pull requests, conventions de commit.

2. Difficultés rencontrées et solutions apportées

Difficulté	Solution apportée
Afficher un seul aperçu de cours à la fois sans JavaScript	Boutons radio cachés partageant le même <code>name</code> , reliés à des <code><label></code> , et sélecteur <code>:checked</code> ~ qui affiche le panneau choisi.
Un navigateur ne peut pas exécuter du C, du Java ou du PHP	Bouton « Executer » qui affiche le résultat attendu, stocké avec chaque exercice.

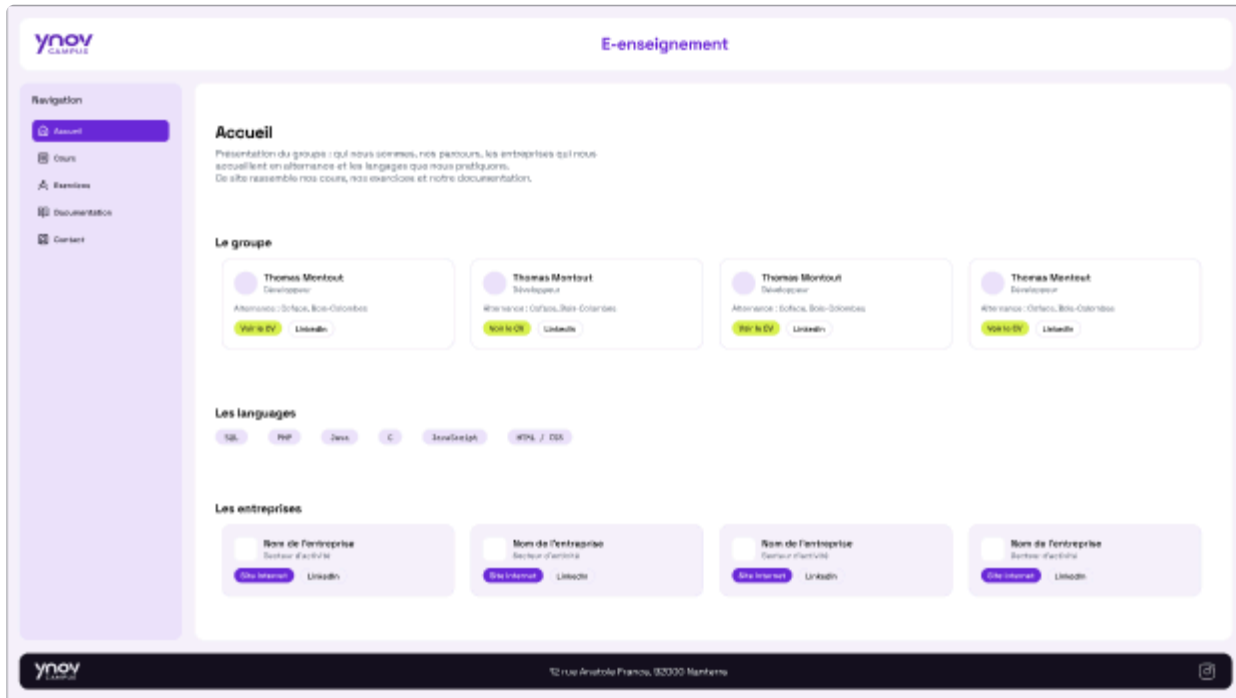
Difficulté	Solution apportée
Afficher du code contenant <code><</code> , <code>></code> et <code>&</code> sans que le navigateur l'interprète	Échappement des caractères dans le HTML des cours, et <code>textContent</code> pour le code des exercices.
Colorer le pseudo-code, inconnu de <code>highlight.js</code>	Déclaration d'un langage « algo » avec ses mots-clés (<code>ALGORITHME</code> , <code>DÉBUT</code> , <code>SI</code> , <code>LIRE</code> ...), accents compris.
Garder les choix d'exercice et de langage au rechargement ou au partage	Choix mémorisés dans les paramètres de l'URL, avec valeurs par défaut.
Publier les CV et livrables sans exposer les brouillons du dossier <code>documents/</code>	Exceptions ciblées dans le <code>.gitignore</code> (<code>!documents/cv-*.pdf</code> , <code>!documents/Documentation/</code>).
Travailler à quatre sur les mêmes fichiers	Une branche par fonctionnalité, fusion par pull request et une seule source pour les styles communs.

3. Perspectives d'évolution

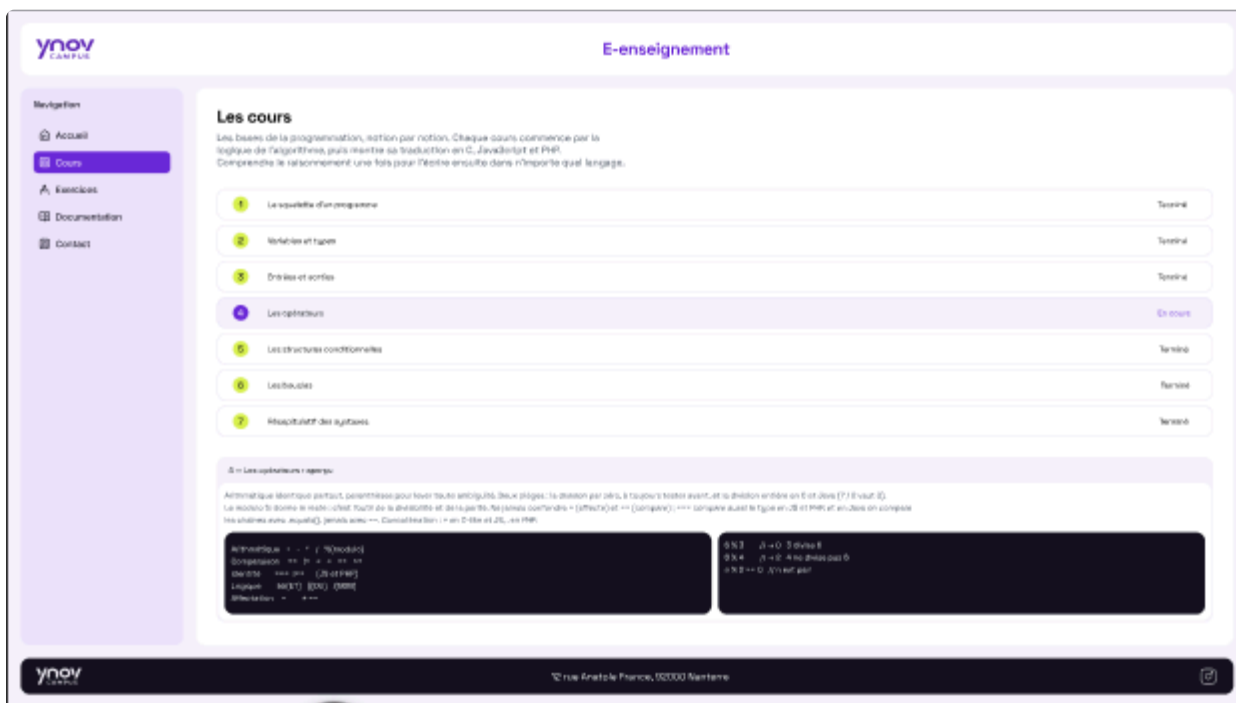
- **Exécution réelle du code** : brancher un service de compilation en ligne pour exécuter les corrigés.
- **Fichiers sources téléchargeables** : proposer chaque corrigé en fichier `.c`, `.java`, `.js` ou `.php`.
- **Enrichissement des contenus** : nouveaux exercices par simple ajout dans `exercices.js`, cours en Java.
- **Suivi de progression** : mémoriser les cours et exercices terminés par l'étudiant.

XI. Annexes

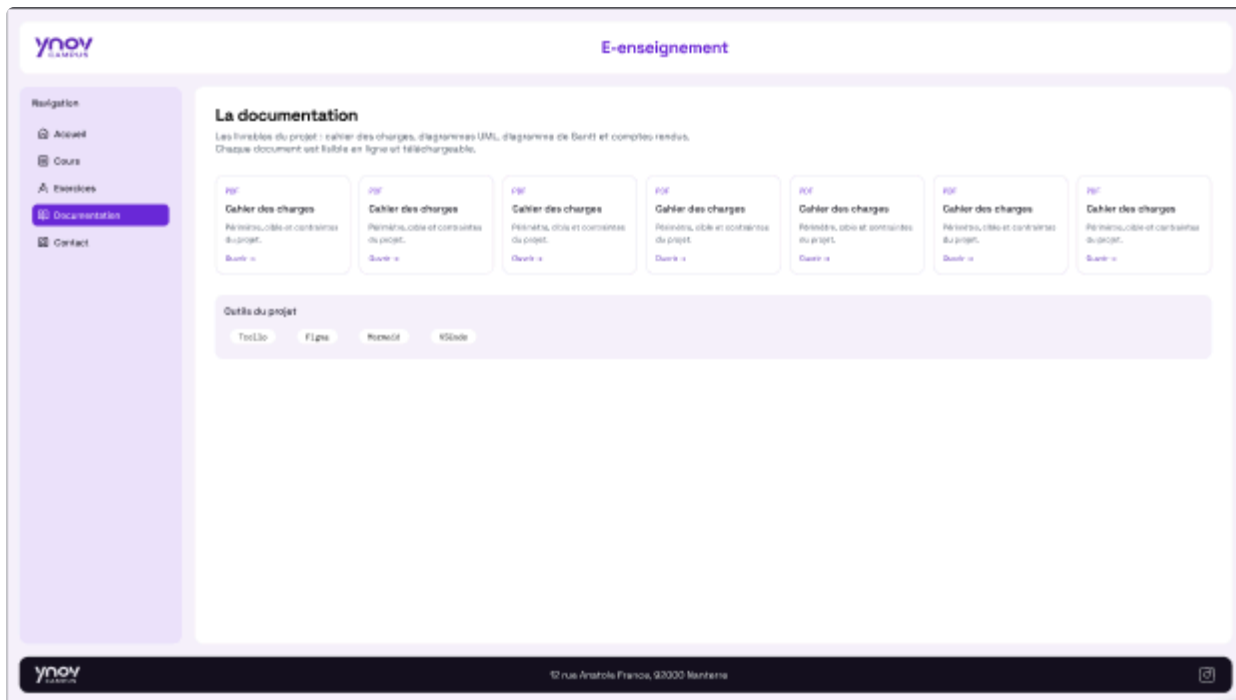
Annexe 1 : Maquettes et Ergonomie (UI/UX)



Page Accueil : présentation du groupe, langages pratiqués et entreprises d'accueil.



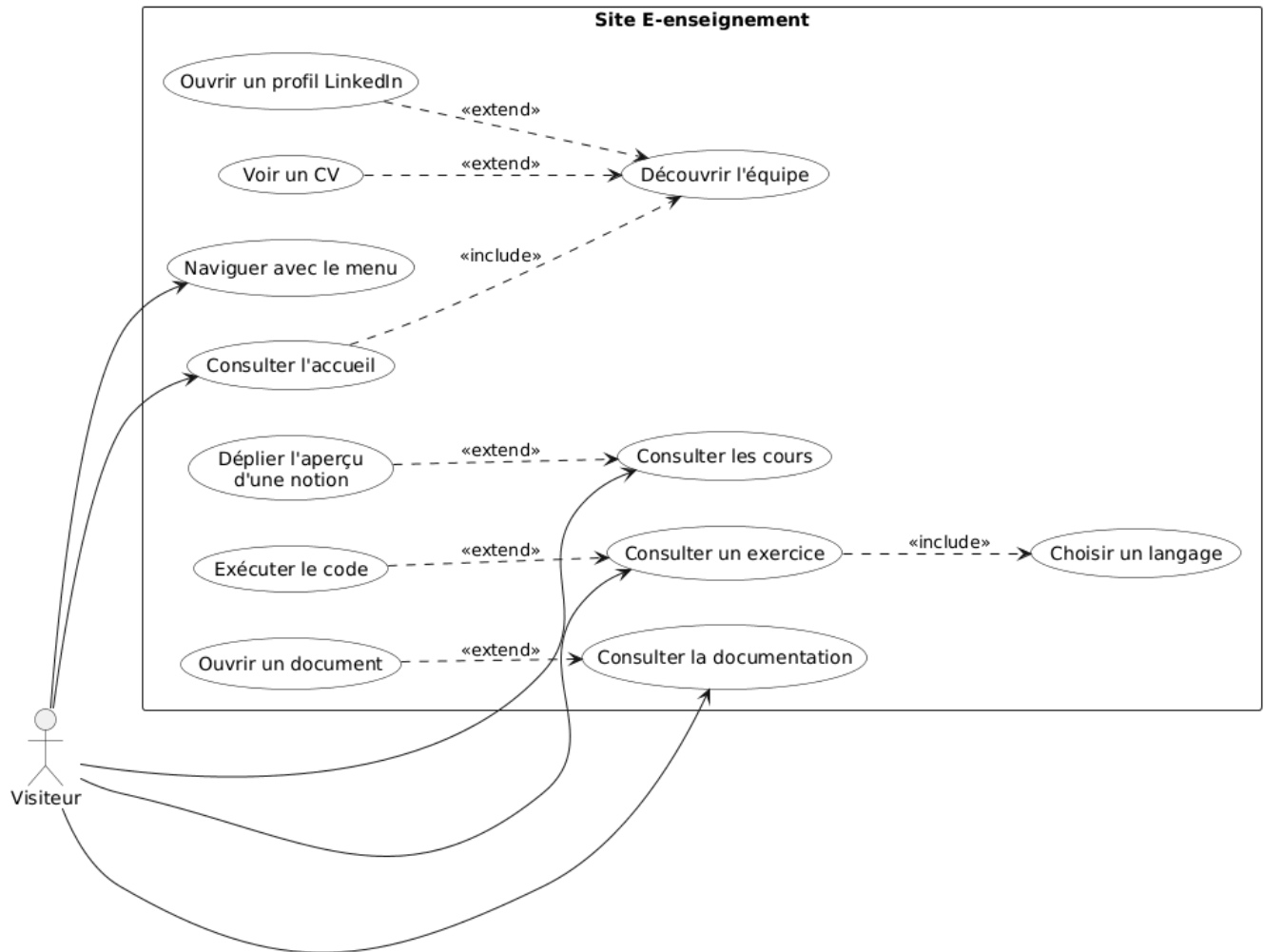
Page Cours : liste numérotée des notions et aperçu déplié avec ses exemples de code.



Page Documentation : cartes des livrables et bloc des outils du projet.

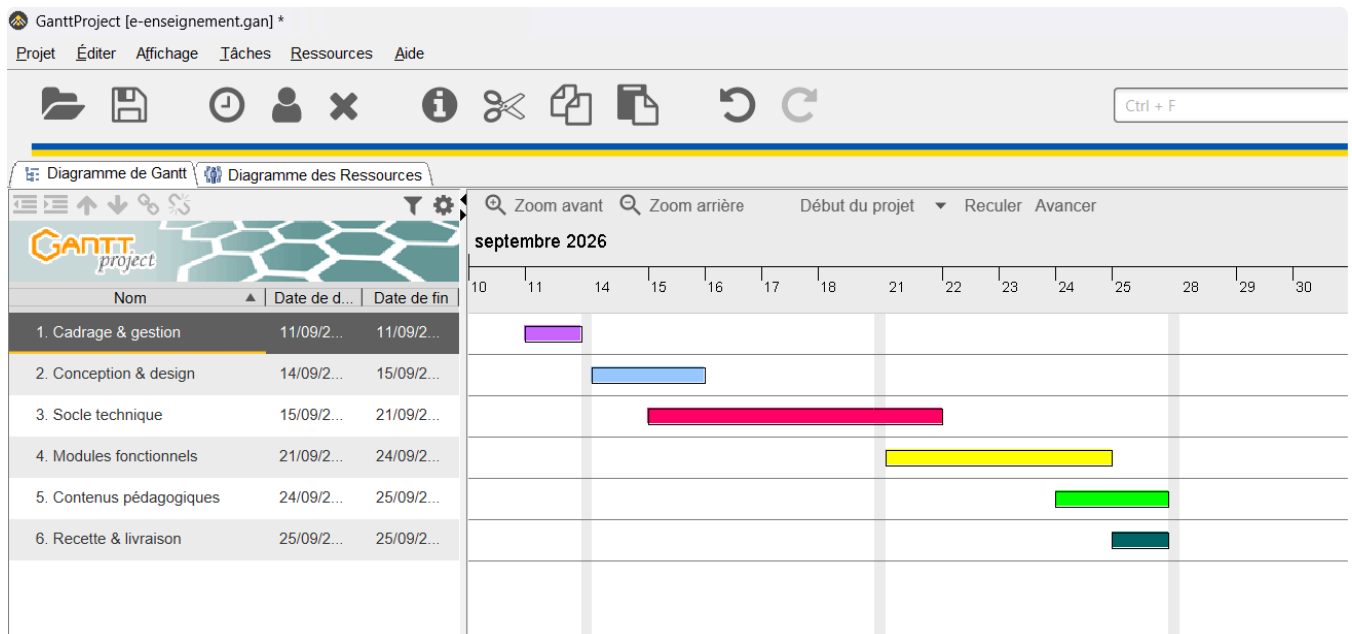
Légende : maquettes réalisées sur Figma avant le développement. Elles fixent le gabarit commun à toutes les pages : barre du haut avec le logo Ynov et le titre du site, menu latéral de navigation sur fond lavande, contenu sur fond blanc organisé en cartes, et pied de page sombre avec l'adresse de l'école.

Annexe 2 : Diagramme de cas d'utilisation



Légende : interactions du visiteur avec le site. Il navigue avec le menu et consulte l'accueil, qui inclut la découverte de l'équipe (étendue par l'ouverture d'un CV ou d'un profil LinkedIn), les cours (dépliage de l'aperçu d'une notion), les exercices (choix d'un langage inclus, exécution du code en extension) et la documentation (ouverture d'un document). Aucun compte n'est nécessaire : tous les visiteurs disposent des mêmes actions.

Annexe 3 : Gestion de Projet et Planification



Légende : diagramme de Gantt réalisé avec GanttProject. Il présente les six lots du projet, du cadrage le 11 septembre 2026 à la recette et la livraison le 25 septembre 2026.

Annexe 4 : Extrait de Code — dépliage des cours en CSS pur

```
<!-- Boutons radio invisibles : un seul coché à la fois (même name) -->
<input class="cours-radio" type="radio" name="cours" id="cours-1" checked />
<input class="cours-radio" type="radio" name="cours" id="cours-2" />
```

```
<!-- Chaque ligne de la liste est un label relié à son bouton radio -->
<label class="cours-ligne" for="cours-1">
  <span class="cours-numero">1</span>
  <span class="cours-titre">Le squelette d'un programme</span>
</label>
```

```
.cours-panneau {
  display: none;
}
```

```
/* Le bouton coché affiche le panneau correspondant */
#cours-1:checked ~ .cours-panneaux #panneau-1,
#cours-2:checked ~ .cours-panneaux #panneau-2 {
  display: block;
  animation: apparition 0.3s ease;
}
```

Légende : extrait de `pages/cours.html` et `css/cours.css`. Le clic sur un `<label>` coche le bouton radio associé ; le sélecteur `:checked ~` affiche alors le bon panneau, sans aucune ligne de JavaScript.

Annexe 5 : Extrait de Code — lecture de l'URL sur la page Exercices

```
// C'est l'URL qui mémorise le langage et l'exercice choisis :  
// on peut recharger la page ou partager le lien sans rien perdre.  
const params = new URLSearchParams(window.location.search);  
  
let language = params.get("language");  
if (!langages[language]) {  
    language = "Algo";  
}  
  
let numeroExercice = Number(params.get("exo"));  
if (!exercices[numeroExercice]) {  
    numeroExercice = 1;  
}  
  
const exerciceSelectionne = exercices[numeroExercice];
```

Légende : extrait de `js/exercices.js`. Les paramètres de l'adresse déterminent l'exercice affiché ; une valeur absente ou inconnue est remplacée par une valeur par défaut, ce qui évite toute erreur d'affichage.

Annexe 6 : Historique de Versionnement et Travail Collaboratif (Git)

Date	Auteur	Message de commit
24/09/2026	Thomas MONTOUT	feat(cours): add arrays, functions and files lessons
24/09/2026	Thomas MONTOUT	feat(exercices): add exercises 04 to 06
24/09/2026	Thomas MONTOUT	style(global): terminal-style site title with blinking cursor
18/09/2026	Thomas MONTOUT	refactor(global): readme update, gitignore update, site name bigger
17/09/2026	Christophe ALEIXO	Merge pull request #6 from FlytziTv/document
17/09/2026	Thomas MONTOUT	style(global): uniformisation of the all pages (typo, animation etc)

Date	Auteur	Message de commit
17/09/2026	Moncef MALLEM	Pages Accueil et Cours en HTML/CSS
17/09/2026	Thomas MONTOUT	feat(exercice): logic of the program exercice.js done
16/09/2026	Thomas MONTOUT	(feat): add exercices page and update layout
15/09/2026	Alexis DE JESUS	Ajout du fichier layout.css et mise à jour de index.html pour la navigation
15/09/2026	Alexis DE JESUS	Ajout du fichier global.css et mise à jour des liens CSS
14/09/2026	Christophe ALEIXO	Setup docker/environnement to work
11/09/2026	Alexis DE JESUS	Ajout des fichiers de base pour le site E5

Légende : sélection de commits du dépôt `FlytziTv/e5-website`, du plus récent au plus ancien. Elle illustre le travail sur des branches dédiées, l'intégration par pull requests et la répartition des responsabilités.